

Feature Summary

Economics Data Suite

A comprehensive technical and intuitive guide to every feature in the repository:
providers, recipes, transforms, models, visualization, and infrastructure.

February 15, 2026

Contents

1	Executive Overview	3
2	Architecture & Data Flow	3
3	Configuration & Feature Gating	4
4	Getting Started (Runbook)	4
4.1	Install and run	4
4.2	What “optional extras” mean	4
4.3	Where outputs live	5
5	Core Concepts (the minimal vocabulary)	5
6	High-Level Methods Overview (why these features exist)	5
6.1	Macro transforms (inflation, growth, spreads, real rates)	5
6.2	Econometrics (OLS, AR(1), local projections/IRFs)	6
6.3	Forecasting (baselines, statistical models, and ML lags)	6
6.4	Technical indicators and news sentiment	6
7	Worked Examples (end-to-end workflows)	6
7.1	Example A: CPI to YoY inflation to plot	6
7.2	Example B: a series to baseline forecasts to ARIMA	7
7.3	Example C: build a regression dataset, run OLS, interpret	7
8	Glossary (quick definitions)	7
9	Reference: Feature Index	8
10	Feature Writeups	10
10.1	Data Ingestion	10
10.1.1	YFinanceProvider	10
10.1.2	FMPPProvider	10
10.1.3	FREDProvider	11
10.1.4	GDELTPProvider	12
10.2	Infrastructure	13
10.2.1	ProviderRegistry	13
10.2.2	Provider base protocol	13
10.3	Forecasting	14
10.3.1	NaiveForecastRecipe	14
10.3.2	DriftForecastRecipe	15
10.3.3	ETSForecastRecipe	15
10.3.4	ARIMAForecastRecipe	16
10.3.5	RidgeLagForecastRecipe	17
10.3.6	RandomForestLagForecastRecipe	17
10.3.7	Forecast utilities	18
10.4	Macro Transforms	19
10.4.1	InflationYoYRecipe	19
10.4.2	InflationMoMAnnRecipe	19
10.4.3	GrowthQoQAnnRecipe	20
10.4.4	SpreadRecipe	20

10.4.5	RealRateRecipe	21
10.5	Econometrics	21
10.5.1	OLSRecipe	21
10.5.2	AR1Recipe	22
10.5.3	LocalProjectionIRFRecipe	23
10.6	News Processing	24
10.6.1	FinBERTDailySentimentRecipe	24
10.7	Technical Indicators	24
10.7.1	SimpleMovingAverageRecipe	24
10.7.2	ExponentialMovingAverageRecipe	25
10.7.3	RSIRecipe	25
10.7.4	MACDRecipe	26
10.7.5	RollingVolatilityRecipe	26
10.8	Time Series Transforms	27
10.8.1	LogReturnsRecipe	27
10.8.2	RollingZScoreRecipe	27
10.9	Infrastructure (continued)	28
10.9.1	RecipeRegistry	28
10.9.2	Recipe base protocol	28
10.9.3	SQLiteStorage	29
10.9.4	JobRunner	29
10.9.5	Pipeline submission	30
10.10	Datasets	30
10.10.1	build_merged_dataset	30
10.11	Visualization	31
10.11.1	plot_timeseries	31
10.11.2	build_recession_intervals	32
10.12	UI & Config	32
10.12.1	Time series catalog	32
10.12.2	AppConfig / load_config	32
10.12.3	Optional dependency helpers	33
10.12.4	Main Streamlit app	33
10.12.5	Legacy pages	34
10.12.6	E2E smoke script	34
11	Completeness Checklist	35

1 Executive Overview

This repository is an “economics data suite”: a small application that helps you (1) **pull time-series data**, (2) **transform it** into more meaningful measures (growth, inflation, spreads, returns), (3) **run simple econometric models** (OLS, AR(1), local projections), and (4) **forecast** and **visualize** the results.

Local-first means everything runs on your computer (no server you have to deploy). The user interface is built in **Streamlit**, which is a Python tool for making simple interactive web apps. Under the hood, the project organizes work as **jobs**: you submit a request (“fetch CPI from FRED” or “compute YoY inflation”), the job runs, and the output is saved as a CSV file you can plot or download.

Non-technical intuition

Think of the app as a “time-series lab notebook.” You choose a series (like CPI, unemployment, or a stock price), press run, and the app saves a clean dataset plus a record of what you did. You can then build up from raw data to transformed series to models and forecasts.

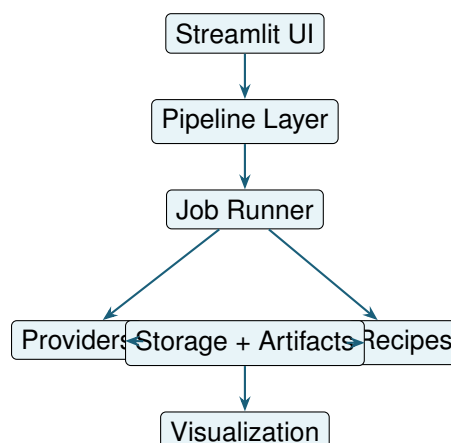
Two main modes:

- **Finance:** market tickers (prices/volume), technical indicators, and forecasting tools.
- **Econ:** FRED macro series, macro transforms, and econometric models.

Dataset Builder: a UI that merges multiple series into one aligned dataset (same date column), which is usually the first step before running regressions.

What this document is: a guided explanation of the data flow and the main methods, followed by a per-feature reference (each provider/recipe) with code snippets.

2 Architecture & Data Flow



Flow (big picture): UI submits provider fetches and recipe runs → JobRunner executes (in background threads) → Providers fetch raw data and Recipes transform/model it → SQLite stores job status/logs, and CSV artifacts store results → Plotly renders charts (optionally with recession shading).

Step-by-step: what happens when you click “Run”:

1. **You choose an input:** a ticker (Finance) or a FRED series ID (Econ), plus dates and options.

2. **The app creates a job:** a new row in the SQLite database with status `QUEUED`.
3. **A worker executes the job:** the `JobRunner` calls either a provider (to fetch data) or a recipe (to transform/model an existing dataset).
4. **Results are saved:** if the job succeeds, its output `DataFrame` is written to a CSV file (an “artifact”) in the results directory.
5. **You inspect and plot:** the UI reads the artifact, renders charts, and lets you download the CSV.

Non-technical intuition

The job system is there so the UI stays responsive. Instead of doing everything immediately in the button click, the app records “what to do” and runs it in the background, then updates the results when finished.

3 Configuration & Feature Gating

Env Var	Purpose
<code>FINREC_DB_PATH</code>	SQLite database path (default: <code>data/finrec.db</code>)
<code>FINREC_RESULTS_DIR</code>	CSV artifact output directory (default: <code>data/results</code>)
<code>FINREC_FMP_API_KEY</code>	API key for FMP provider (required when using fmp)

Optional extras (`pyproject.toml`): `dev`, `market`, `macro`, `news`, `ml`, `econ`, `forecast`. Providers and recipes that depend on optional packages are registered only when those packages are installed; runtime calls use `require_optional()` and raise a friendly install hint if missing.

4 Getting Started (Runbook)

Goal: get the Streamlit UI running locally, then run a first end-to-end workflow.

4.1 Install and run

1. Create and activate a Python environment.
2. Install the package (and optional extras if you want specific features).
3. Create a `.env` file (copy from `.env.example`) if you want to change default paths or add an API key.
4. Start Streamlit: `streamlit run streamlit_app.py`.

4.2 What “optional extras” mean

Some features require heavier libraries (for example `statsmodels` for econometrics or `transformers` for FinBERT sentiment). To keep installation light, those are optional. If you try to use a feature without its dependency installed, the app will raise a message telling you which extra to install, e.g.:

```
pip install -e "[forecast]"
```

4.3 Where outputs live

- **SQLite database:** stores job metadata, status, and logs (default: `data/finrec.db`).
- **CSV artifacts:** each successful job writes `{job_id}.csv` to the results directory (default: `data/results/`).

5 Core Concepts (the minimal vocabulary)

Time series

A dataset where each observation has a date/time (monthly CPI, daily stock prices, quarterly GDP).

Provider

A data source connector that *fetches raw data* (market prices, FRED macro series, news articles).

Recipe

A transformation or model that *takes a dataset* and produces a new dataset (inflation, returns, OLS results, forecasts).

Job A run of a provider or recipe, tracked with an ID, status, logs, and (if it succeeds) an output CSV artifact.

Artifact

The saved output CSV from a job. Artifacts are the main “inputs” to later steps.

Non-technical intuition

If you remember one idea: providers → recipes → artifacts. You start with raw series, compute transformed series, and then run models/forecasts on those transformed series.

6 High-Level Methods Overview (why these features exist)

6.1 Macro transforms (inflation, growth, spreads, real rates)

Economists often transform raw levels into rates of change or differences. For example:

- **Inflation** is the growth rate of a price index (CPI). Year-over-year rates help smooth seasonal noise.
- **Growth rates** of output (GDP) are often annualized to make numbers comparable across frequencies.
- **Spreads** (like 10-year minus 2-year yields) summarize information about the yield curve.
- **Real rates** subtract inflation from nominal rates to approximate inflation-adjusted returns.

These transforms are implemented as recipes so you can compute them consistently and save the result as a new artifact.

6.2 Econometrics (OLS, AR(1), local projections/IRFs)

This app includes a small set of “workhorse” models:

- **OLS** estimates a linear relationship between a dependent variable and one or more predictors.
- **AR(1)** is the simplest persistence model for a single series: today depends on yesterday.
- **Local projections (IRFs)** estimate how a variable responds over time to a shock.

Important: these tools help you explore and learn, but they do not automatically solve identification or causal inference. Interpretation should be cautious.

6.3 Forecasting (baselines, statistical models, and ML lags)

Forecasting is useful when you want a simple quantitative guess about future values, or when you want to compare methods. The suite includes:

- **Baselines** (naive, drift): fast, easy benchmarks.
- **Statistical models** (ETS, ARIMA): common time-series forecasting methods with uncertainty intervals.
- **Lag-based ML** (ridge, random forest): supervised learning on lagged values; can capture nonlinear patterns but can also overfit.

6.4 Technical indicators and news sentiment

- **Technical indicators** (SMA/EMA/RSI/MACD/volatility) summarize price momentum, trend, and variability. They are descriptive features, not “proof” of predictability.
- **News sentiment** converts article text into a numeric score (positive/negative) and aggregates it by day, so you can treat it like another time series.

7 Worked Examples (end-to-end workflows)

7.1 Example A: CPI to YoY inflation to plot

Question: “How has inflation evolved over time?” A common starting point is CPI inflation year-over-year.

1. In **Econ** mode, fetch CPI from FRED (e.g., CPIAUCSL) over a long date range.
2. Run the **InflationYoYRecipe** on the fetched CPI artifact.
3. Plot the resulting YoY series and compare peaks/troughs across time.

Non-technical intuition

YoY inflation answers: “Compared to the same month last year, how much higher is the price level today?” This helps avoid misleading month-to-month noise.

7.2 Example B: a series to baseline forecasts to ARIMA

Question: “If I had to project this series forward, what would simple methods say?”

Workflow:

1. Choose a target series (macro series or a price series).
2. Run **Naive** and **Drift** forecasts first. These are baselines: if a fancy model cannot beat them, that is a warning sign.
3. Run **ARIMA**. Look at the forecast path and (if provided) the confidence interval to understand uncertainty.

Non-technical intuition

Forecast evaluation is hard. This app’s goal is to make it easy to compare methods in a consistent pipeline, not to claim any one method is “best” in general.

7.3 Example C: build a regression dataset, run OLS, interpret

Question: “How does inflation relate to interest rates (in this sample)?”

This example emphasizes workflow and interpretation rather than causal claims.

1. Fetch two series from FRED (for example, CPI to build inflation and a nominal rate series).
2. Use recipes to transform them into comparable units (e.g., inflation YoY and a nominal rate in percent).
3. Use the **Dataset Builder** to merge the two artifacts into one dataset with aligned dates.
4. Run **OLSRecipe** with inflation as y and the nominal rate as x (or vice versa).
5. Interpret the coefficient as an **association** within the sample and check the standard error and p-value for uncertainty.

Non-technical intuition

OLS can tell you “when x is higher, does y tend to be higher too?” But it does not automatically tell you why. For causal interpretation, you need a research design (identification) beyond a simple regression.

8 Glossary (quick definitions)

OHLCV

Open, High, Low, Close, Volume. A common daily stock-price dataset format.

Return

A change in value from one period to the next. In finance, returns are often modeled instead of prices.

Log return

$\ln(p_t) - \ln(p_{t-1})$. Approximately the percent change (in decimals) for small changes.

Annualized

Scaled to “per year” units. Example: monthly log change $\times 12 \times 100$ becomes annualized percent.

Lag A past value of a series, like y_{t-1} .

Horizon

How far ahead you forecast or measure a response (e.g., 6 months ahead).

Train/test split

A common forecasting workflow: fit the model on earlier data (train) and evaluate on later data (test).

Stationary

Roughly: a series whose statistical properties (mean/variance) do not change over time. Many models behave better on stationary series.

AIC Akaike Information Criterion. A metric that trades off fit vs model complexity; lower is better in-sample under the criterion.

Standard error

An estimate of the uncertainty (sampling variability) of a coefficient estimate.

p-value

Under the model assumptions, the probability of seeing a t -statistic as extreme as observed if the true coefficient were 0.

Confidence interval

A range of plausible values for a parameter or forecast, under model assumptions.

IRF Impulse Response Function: the response of a variable over time after a one-time shock.

9 Reference: Feature Index

#	Feature	Category	Ref
1	YFinanceProvider	Data Ingestion	10.1.1
2	FMPPProvider	Data Ingestion	10.1.2
3	FREDProvider	Data Ingestion	10.1.3
4	GDELTProvider	Data Ingestion	10.1.4
5	ProviderRegistry	Infra	10.2.1
6	Provider base protocol	Infra	10.2.2
7	NaiveForecastRecipe	Forecasting	10.3.1
8	DriftForecastRecipe	Forecasting	10.3.2
9	ETSForecastRecipe	Forecasting	10.3.3
10	ARIMAForecastRecipe	Forecasting	10.3.4
11	RidgeLagForecastRecipe	Forecasting	10.3.5
12	RandomForestLagForecastRecipe	Forecasting	10.3.6
13	Forecast utilities	Infra	10.3.7
14	InflationYoYRecipe	Macro Transform	10.4.1
15	InflationMoMAnnRecipe	Macro Transform	10.4.2

Continued on next page

Continued from previous page

#	Feature	Category	Ref
16	GrowthQoQAnnRecipe	Macro Transform	10.4.3
17	SpreadRecipe	Macro Transform	10.4.4
18	RealRateRecipe	Macro Transform	10.4.5
19	OLSRecipe	Econometrics	10.5.1
20	AR1Recipe	Econometrics	10.5.2
21	LocalProjectionIRFRecipe	Econometrics	10.5.3
22	FinBERTDailySentimentRecipe	News	10.6.1
23	SimpleMovingAverageRecipe	Technical	10.7.1
24	ExponentialMovingAverageRecipe	Technical	10.7.2
25	RSIRecipe	Technical	10.7.3
26	MACDRecipe	Technical	10.7.4
27	RollingVolatilityRecipe	Technical	10.7.5
28	LogReturnsRecipe	Time Series	10.8.1
29	RollingZScoreRecipe	Time Series	10.8.2
30	RecipeRegistry	Infra	10.9.1
31	Recipe base protocol	Infra	10.9.2
32	SQLiteStorage	Storage	10.9.3
33	JobRunner	Orchestration	10.9.4
34	Pipeline submission	Orchestration	10.9.5
35	build_merged_dataset	Datasets	10.10.1
36	plot_timeseries	Visualization	10.11.1
37	build_recession_intervals	Visualization	10.11.2
38	Time series catalog	UI	10.12.1
39	AppConfig / load_config	Config	10.12.2
40	Optional dependency helpers	Infra	10.12.3
41	Main Streamlit app	UI	10.12.4
42	Legacy pages	UI	10.12.5
43	E2E smoke script	Scripts	10.12.6

10 Feature Writeups

10.1 Data Ingestion

10.1.1 YFinanceProvider

What it is: Fetches market OHLCV data via the yfinance library (Yahoo Finance).

Technical: Uses `yfinance.Ticker(symbol).history()` with date range or fallback `n` rows. Rate limiting: global throttle (default 3s) and retry with backoff on `YFRateLimitError`. Output: date, symbol, open, high, low, close, volume.

When to use:

- When you want a free, quick way to pull daily price history for common tickers.
- When you want a fallback source if an API provider rejects a symbol (e.g., paywalled endpoints elsewhere).

Inputs/outputs (practical):

- **Input:** a ticker symbol (e.g., AAPL) and a date range (or a row limit).
- **Output:** one row per trading day with OHLCV (Open, High, Low, Close, Volume).

Common pitfalls:

- **Rate limits** happen. If you request too often, Yahoo can temporarily block you; the provider sleeps and retries.
- **Non-trading days:** there is no data on weekends/holidays, so daily series have gaps.
- **Interpretation:** OHLC are prices; to model/forecast, you often want returns (see [10.8.1](#)) rather than raw levels.

Where: `src/finrec/providers/market/yfinance.py` — YFinanceProvider

```
@dataclass
class YFinanceProvider(Provider):
    meta: ProviderMeta = ProviderMeta(
        id="yfinance", name="yfinance", kind="market",
    )
    def fetch(self, request: dict, ctx) -> pd.DataFrame:
        yf = require_optional("yfinance", extra_hint="market")
```

Non-technical intuition

This is the “Google it for me” option for market data: give it a ticker and dates and it downloads price history. Because Yahoo sometimes blocks frequent requests, the provider intentionally waits between calls and retries if it gets rate-limited.

10.1.2 FMPPProvider

What it is: Fetches market OHLCV from Financial Modeling Prep API.

Technical: Stable EOD endpoint; daily bars only. On 402 (premium symbol) errors, falls back to YFinanceProvider when available.

When to use:

- When you have an FMP API key and want a stable HTTP API data source.

- When you want a consistent JSON/HTTP response format (useful for debugging and reproducibility).

Inputs/outputs (practical):

- **Input:** ticker symbol and date range.
- **Output:** daily OHLCV similar to Yahoo Finance.

Common pitfalls:

- **API key required:** set `FINREC_FMP_API_KEY` in `.env`.
- **Premium symbols:** some tickers may return HTTP 402; in that case the app tries Yahoo as a fallback.

Where: `src/finrec/providers/market/fmp.py`

```
@dataclass
class FMPPProvider(Provider):
    meta: ProviderMeta = ProviderMeta(id="fmp", name="FMP", kind="market")
    def fetch(self, request: dict, ctx) -> pd.DataFrame:
        requests = require_optional("requests", extra_hint="market")
        api_key = (os.getenv("FINREC_FMP_API_KEY") or "").strip()
        if not api_key:
            raise ValueError("Missing FINREC_FMP_API_KEY")
        resp = requests.get("https://financialmodelingprep.com/stable/...",
                           params={...})
```

Non-technical intuition

FMP is a “paid-optional” market data provider. If your key can access the symbol, you get a clean daily price history. If the API says the symbol is premium-only, the app tries Yahoo Finance instead (if installed).

10.1.3 FREDProvider

What it is: Fetches macroeconomic time series from FRED via pandas-datareader.

Technical: `DataReader(series_id, "fred", start, end)`. Output: `date, series_id, value`. End date clamped to today.

When to use:

- When you need standard U.S. macro series (CPI, unemployment, Fed funds, GDP, etc.).
- When you want a transparent, citeable data source for class projects.

Inputs/outputs (practical):

- **Input:** `series_id` (like `CPIAUCSL` or `UNRATE`) and a date range.
- **Output:** one row per observation date (monthly/weekly/quarterly depends on the series) with a single numeric value.

Common pitfalls:

- **Frequency matters:** CPI is monthly, GDP is quarterly, unemployment is monthly, etc. This affects annualization formulas and model choices.

- **Revisions:** some macro series get revised over time; your results can change if you re-download later.

Where: `src/finrec/providers/macro/fred.py`

```
@dataclass
class FREDProvider(Provider):
    meta: ProviderMeta = ProviderMeta(id="fred", name="FRED", kind="macro")

    def fetch(self, request: dict, ctx) -> pd.DataFrame:
        pdr_data = require_optional("pandas_datareader.data", extra_hint="macro")
        series_id = str(request.get("series_id", "CPIAUCSL")).upper()
        df = pdr_data.DataReader(series_id, "fred", start, end)
```

Non-technical intuition

FRED is the “default” macro data source for many U.S. econ projects. You give it a series ID and dates, and it returns a clean time series you can immediately transform (inflation/growth) or use in regressions.

10.1.4 GDELTProvider

What it is: Fetches news articles from the GDELT document API.

Technical: Endpoint with `mode=ArtList`, `format=json`. Rate limit: 5s throttle; retry on 429. Supports `sourcelang: filter`.

When to use:

- When you want to build a simple “news intensity” or “news sentiment” time series from article text.
- When you want a broad cross-source news search without hand-curating outlets.

Inputs/outputs (practical):

- **Input:** a query string (keywords, entities) and a date range; optionally a language filter.
- **Output:** a table of articles (date/time, title, snippet, URL, source metadata).

Common pitfalls:

- **Rate limits:** the provider waits and retries to avoid HTTP 429.
- **Query design:** broad queries can return unrelated articles; narrow queries can miss relevant coverage.
- **Selection bias:** “news about X” is not a random sample; interpret any analysis carefully.

Where: `src/finrec/providers/news/gdelt.py`

```
@dataclass
class GDELTProvider(Provider):
    meta: ProviderMeta = ProviderMeta(id="gdelt", name="GDELT (doc API)", kind="news")

    def fetch(self, request: dict, ctx) -> pd.DataFrame:
        requests = require_optional("requests", extra_hint="news")
```

```

    query = _normalize_gdelt_query(str(request.get("query", "inflation
    )))
    resp = requests.get("https://api.gdeltproject.org/api/v2/doc/doc",
    params={...})
    return _parse_articles(resp.json())

```

Non-technical intuition

GDELT lets you “search the news like a database.” You give it a query and dates, and it returns matching articles. The app can then score or aggregate these articles into a daily time series you can compare with economic/financial variables.

10.2 Infrastructure

10.2.1 ProviderRegistry

What it is: Registry for data providers with lazy registration based on optional dependencies.

Why it exists: the UI needs to show a list of available providers (market/macro/news). But not everyone installs every optional dependency. The registry solves this by registering a provider only when its Python package dependencies are available.

How to extend (developer view):

- Implement a new provider class that follows the Provider protocol (see 10.2.2).
- Register it in the registry initialization code (typically inside the “try register” function).
- If it needs a third-party package, gate it behind `require_optional(...)` so users get a clear install message.

```

def get_registry() -> ProviderRegistry:
    global _REGISTRY
    if _REGISTRY is None:
        reg = ProviderRegistry(_providers={})
        _try_register_real_providers(reg)
        _REGISTRY = reg
    return _REGISTRY

```

Non-technical intuition

Central list of data sources; only those whose libraries are installed show up.

10.2.2 Provider base protocol

What it is: Protocol: Provider has meta: ProviderMeta and `fetch(request, ctx) -> DataFrame`. ProviderKind is "market" | "macro" | "news".

Why it matters: this is the “contract” that keeps the pipeline consistent. If every provider returns a DataFrame, then the job runner can save it as a CSV artifact and downstream recipes can consume it.

Common pitfalls:

- Ensure the output includes a date column in a consistent format.

- Avoid returning provider-specific column naming unless necessary; recipes and UI work best with predictable schemas.

Non-technical intuition

A provider is a “data downloader” with a standardized interface. If you can call `fetch(...)` and get a `DataFrame` back, the rest of the pipeline (jobs, artifacts, recipes, plots) can treat all providers the same way.

```
@dataclass(frozen=True)
class ProviderMeta:
    id: str
    name: str
    kind: ProviderKind

class Provider(Protocol):
    meta: ProviderMeta
    def fetch(self, request: dict, ctx: JobContext) -> pd.DataFrame: ...
```

10.3 Forecasting

10.3.1 NaiveForecastRecipe

What it is: Baseline forecast: $\hat{y}_{t+h} = y_T$ for all $h \geq 1$.

When to use:

- As a **benchmark**. Any more complex forecast method should at least be compared to naive.
- When a series is very noisy and has no obvious trend/seasonality (naive can be surprisingly hard to beat).

Inputs/outputs (practical):

- **Input:** a univariate time series (one numeric column over time) and a forecast horizon.
- **Output:** predicted values for the held-out test period (if used) and for future dates, formatted as a standard forecast artifact.

Interpretation: the model is not “learning” anything besides the last observed value. If naive performs similarly to a complex model, that often means your series is hard to forecast with the information you provided.

Where: `src/finrec/recipes/forecast/naive.py`

```
last = float(train_y.dropna().iloc[-1])
test_yhat = [last] * n_test
fcst_yhat = [last] * len(future_dates)
out = build_forecast_output(
    train_dates=train_dates, train_y=train_y,
    test_dates=test_dates, test_y=test_y, test_yhat=test_yhat,
    future_dates=future_dates, fcst_yhat=fcst_yhat,
)
```

Non-technical intuition

Assumes tomorrow will look like today. It is intentionally simple: you use it to sanity-check whether fancier models are adding value.

10.3.2 DriftForecastRecipe

What it is: Linear drift: $\hat{y}_{t+h} = y_T + \frac{y_T - y_0}{T-1} \cdot h$.

When to use:

- As a second baseline when the series has an obvious long-run trend.
- When you want a transparent “straight-line” extrapolation.

Common pitfalls:

- Drift can be badly misleading if the start/end points are unusual (e.g., the sample ends during a temporary spike).
- Like naive, it ignores seasonality and changes in trend over time.

Where: `src/finrec/recipes/forecast/naive.py`

```
drift = (yT - y0) / max(1, (n_train - 1))
def _path(steps: int) -> list[float]:
    return [yT + drift * (k + 1) for k in range(steps)]
test_yhat = _path(n_test)
fcst_yhat = _path(len(future_dates))
```

Non-technical intuition

Draws a straight line from the first observation to the last observation, then keeps going. It is still a baseline, just one that allows a constant trend.

10.3.3 ETSForecastRecipe

What it is: Exponential smoothing (Holt-Winters) with configurable trend and seasonal components. Uses `statsmodels.tsa.holtwinters.ExponentialSmoothing`.

When to use:

- When the series has a smooth level and possibly a trend and/or seasonality (e.g., monthly macro series).
- When you want a classic forecasting method that is relatively robust and interpretable.

How it works (plain English): ETS maintains a smoothed estimate of the current **level** (typical value), optionally a **trend** (direction of movement), and optionally a **seasonal pattern** (repeating ups/downs within the year). New data gets more weight than old data, which is why it is called “exponential” smoothing.

Common pitfalls:

- Seasonality requires the right **seasonal period** (e.g., 12 for monthly data). If you set it wrong, forecasts can look strange.
- ETS is designed for reasonably regular frequencies; irregular date spacing is a warning sign.

Where: `src/finrec/recipes/forecast/ets.py`


```

model = sm_hw.ExponentialSmoothing(
    y_train, trend=trend, seasonal=seasonal, seasonal_periods=
    seasonal_periods
)
res = model.fit(optimized=True)
test_yhat = res.forecast(steps=n_test)
fcst_yhat = res.forecast(steps=len(future_dates))

```

Non-technical intuition

Smooths the series to estimate “where we are now” (level), “where we are heading” (trend), and sometimes “what time of year it is” (seasonality), then extrapolates forward.

10.3.4 ARIMAForecastRecipe

What it is: ARIMA(p, d, q) with AIC-based order selection. Model:

$$y_t = c + \phi_1 y_{t-1} + \dots + \phi_p y_{t-p} + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}$$

Grid search over (p, d, q); selects lowest AIC. Produces confidence intervals.

What p, d, q mean:

- p : how many lagged values $y_{t-1}, y_{t-2}, \dots$ the model uses (autoregressive part).
- d : how many times the series is differenced to remove trends (to make it closer to “stationary”).
- q : how many lagged forecast errors the model uses (moving-average part).

When to use:

- When you want a standard time-series model that can handle persistence and some types of trends via differencing.
- When you want forecast intervals (a rough sense of uncertainty), not just a point forecast.

Common pitfalls:

- **Order selection** is not magic: AIC is a tradeoff between fit and complexity, but it does not guarantee best out-of-sample performance.
- **Small samples** can make ARIMA unstable (estimation errors, failed fits, wide intervals).
- **Structural breaks** (policy changes, crises) can make any historical model misleading.

Where: `src/finrec/recipes/forecast/arima.py`

```

orders = list(itertools.product(list(p_vals), list(d_vals), list(q_vals)))
for (p, d, q) in orders:
    try:
        model = sm_sarimax.SARIMAX(y_train, order=(p, d, q), trend="c",
        ...)
        res = model.fit(dispatch=False)
        if float(res.aic) < best_aic:

```

```

        best_aic, best_res, best_order = float(res.aic), res, (p, d, q)

    except Exception:
        continue

```

Non-technical intuition

ARIMA is a flexible model for “this series depends on its own past.” The recipe tries a few combinations of p, d, q , picks the one with the best AIC, and then forecasts forward with uncertainty bands.

10.3.5 RidgeLagForecastRecipe

What it is: Ridge regression on lagged values. Features: last lookback values plus optional rolling mean/std. Recursive multi-step forecast.

When to use:

- When you want a simple ML-style forecast that treats “lags as predictors” in a supervised learning setup.
- When you suspect the best linear predictor uses multiple recent lags (not just one).

How it works (plain English): build a training dataset where each row is “the last L observations” and the label is “the next observation.” Fit a linear model with ridge regularization (shrinks coefficients to reduce overfitting). To forecast multiple steps, predict one step ahead, append it, then repeat.

Common pitfalls:

- **Recursive forecasts** can drift because the model feeds its own predictions back as inputs.
- **Overfitting:** even a regularized model can overfit if you have too many features relative to data length.
- **Levels vs returns:** predicting price levels can be harder than predicting returns; consider transforms first.

Where: `src/finrec/recipes/forecast/ml_lags.py`

```

X_train, y_train_sup = _build_supervised(train_y, lookback=lookback,
    include_stats=include_stats)
model = sk_pipe.make_pipeline(sk_pre.StandardScaler(), sk_linear.Ridge(
    alpha=alpha))
model.fit(X_train, y_train_sup)
yhat_next = float(model.predict(X_last.reshape(1, -1))[0])

```

Non-technical intuition

Turns the time series into a “predict next value from recent history” dataset, fits a regularized linear model, then rolls forward step-by-step to build a forecast path.

10.3.6 RandomForestLagForecastRecipe

What it is: Same lag features as Ridge; uses RandomForestRegressor instead.

When to use:

- When you suspect nonlinear patterns in the relationship between recent lags and the next value.
- When you want a flexible model that can fit interactions without hand-specifying them.

Common pitfalls:

- **Data-hungry:** random forests generally need more data than simple linear models to generalize well.
- **Interpretability:** forecasts can be harder to explain than ETS/ARIMA.
- Like ridge, it uses **recursive** multi-step forecasting, which can accumulate error.

Where: `src/finrec/recipes/forecast/ml_lags.py`

```
X_train, y_train_sup = _build_supervised(train_y, lookback=lookback,
    include_stats=include_stats)
model = sk_ens.RandomForestRegressor(n_estimators=n_estimators, max_depth=
    max_depth, random_state=0)
model.fit(X_train, y_train_sup)
yhat_next = float(model.predict(X_last.reshape(1, -1))[0])
```

Non-technical intuition

Same idea as ridge (predict next value from recent lags), but with a collection of decision trees. This can fit nonlinear relationships, but can also overfit if the dataset is small.

10.3.7 Forecast utilities

What it is: `build_target_series`, `split_train_test`, `build_forecast_output`, `_infer_step`, `_make_future_dates` in `src/finrec/recipes/forecast/_utils.py`.

Why they matter: these helper functions make all forecast recipes produce output in the same format. That consistency is important for the UI: it can plot any forecast artifact without having to special-case each method.

How to read forecast outputs (typical):

- A forecast artifact usually contains a date column, the actual value (if available), the predicted value, and a segment label (train/test/forecast).
- If confidence intervals are available, they show a range of plausible values given the model assumptions.

Non-technical intuition

Forecasting recipes can be very different internally (ARIMA vs random forest), but the utilities make them *look the same* to the UI: same columns, same segments, same plotting behavior.

```
def split_train_test(dates, y, *, test_size):
    n = len(y)
    n_test = int(math.floor(n * test_size)) if isinstance(test_size, float)
    else int(test_size)
    n_test = max(0, min(n_test, n - 5)) # leave at least 5 in train
    return dates[:-n_test], y.iloc[:-n_test], dates[-n_test:], y.iloc[-
n_test:]
```

10.4 Macro Transforms

10.4.1 InflationYoYRecipe

What it is: Year-over-year inflation:

$$\text{inflation_yoy}_t = 100 \times \left(\frac{x_t}{x_{t-12}} - 1 \right)$$

When to use:

- When x_t is a **monthly price index** (like CPI) and you want an annual inflation rate.
- When you want to reduce seasonality/noise compared to month-over-month changes.

Interpretation: if inflation_yoy is 3, the index is about 3% higher than it was one year earlier (same month last year).

Common pitfalls:

- This formula assumes a **12-period lag** corresponds to one year; that is appropriate for monthly data but not for quarterly/weekly data.
- If the series has missing values, the first 12 observations cannot have YoY inflation.

Non-technical intuition

Compares each month to the same month a year ago to get annual inflation.

Where: src/finrec/recipes/macro/inflation_yoy.py

```
x = pd.to_numeric(df[value_col], errors="coerce")
denom = x.shift(12)
yoy = 100.0 * (x / denom - 1.0)
yoy = yoy.replace([float("inf"), float("-inf")], pd.NA)
df2[out_col] = yoy
```

10.4.2 InflationMoMAnnRecipe

What it is: Annualized MoM: $\text{inflation_mom_ann}_t = 1200 \times (\ln x_t - \ln x_{t-1})$.

Why the factor 1200? For small changes, $\ln x_t - \ln x_{t-1}$ is approximately the monthly percent change in **decimal** form. Multiplying by 100 converts to percent, and multiplying by 12 annualizes a monthly rate: $12 \times 100 = 1200$.

When to use:

- When x_t is monthly and positive (logs require $x_t > 0$).
- When you want an “if this month’s inflation continued for a year” style annualized rate.

Common pitfalls:

- Month-to-month annualized inflation can be very volatile; it is often used for short-run dynamics, not long-run summaries.
- Do not use on quarterly data unless you adjust the annualization factor.

Non-technical intuition

Takes the month-to-month change in a price index and expresses it as an annual rate. It is like saying: “if this month’s pace kept going for 12 months, what would the annual inflation rate be?”

Where: `src/finrec/recipes/macro/inflation_mom_ann.py`

```
x = pd.to_numeric(df[value_col], errors="coerce")
x = x.where(x > 0)
mom_ann = 1200.0 * (np.log(x) - np.log(x.shift(1)))
df2[out_col] = mom_ann
```

10.4.3 GrowthQoQAnnRecipe

What it is: Annualized QoQ: $\text{growth_qoq_ann}_t = 400 \times (\ln x_t - \ln x_{t-1})$.

Why the factor 400? Quarterly log growth is multiplied by 4 to annualize and by 100 to convert to percent: $4 \times 100 = 400$.

When to use:

- When x_t is quarterly and positive (e.g., real GDP).
- When you want growth rates reported in the standard “annualized percent” macro style.

Common pitfalls:

- If you use monthly data here, the annualization will be wrong (you would need $\times 1200$ for monthly log growth).
- As with inflation, annualized short-horizon growth can be noisy.

Non-technical intuition

Quarter-to-quarter growth expressed as an annual rate. This is the style you often see in macro releases: a quarterly change scaled up to “per year” units.

Where: `src/finrec/recipes/macro/growth_qoq_ann.py`

```
x = pd.to_numeric(df[value_col], errors="coerce")
x = x.where(x > 0)
growth = 400.0 * (np.log(x) - np.log(x.shift(1)))
df2[out_col] = growth
```

10.4.4 SpreadRecipe

What it is: Spread $A - B$ between two columns.

When to use:

- To construct yield spreads (e.g., 10-year minus 2-year Treasury yields) or any “difference” measure.
- To compare two related series in the same units (both in percent, both in index points, etc.).

Interpretation: spreads summarize relative movements. For yields, a negative long-minus-short spread is often discussed as “inversion,” but interpretation depends on context.

Non-technical intuition

A spread is a simple “difference” feature. For example, (10Y yield) minus (2Y yield) compresses the yield curve into one number you can plot, regress on, or forecast.

Where: `src/finrec/recipes/macro/spread.py`

```
a = pd.to_numeric(df[series_a], errors="coerce")
b = pd.to_numeric(df[series_b], errors="coerce")
df2[out_col] = a - b
```

10.4.5 RealRateRecipe

What it is: Ex-post real rate: $\text{real_rate} = \text{nominal_rate} - \text{inflation_yoy}$.

When to use:

- When you have a nominal interest rate series (e.g., a policy rate or yield) and an inflation measure.
- When you want a rough inflation-adjusted rate for descriptive analysis.

Common pitfalls:

- This is **ex-post** (uses realized inflation). “Expected” real rates would require an expectations measure.
- Make sure the inflation series is comparable (frequency and units). If nominal is in percent, inflation should be in percent too.

Non-technical intuition

Real rates are a “purchasing power” idea: nominal rates tell you how dollars grow; subtracting inflation approximates how purchasing power grows.

Where: `src/finrec/recipes/macro/real_rate.py`

```
nominal = pd.to_numeric(df[nominal_col], errors="coerce")
infl = pd.to_numeric(df[inflation_col], errors="coerce")
df2[out_col] = nominal - infl
```

10.5 Econometrics

10.5.1 OLSRecipe

What it is: OLS regression $y = X\beta + \varepsilon$ via `statsmodels.api.OLS`. Output: `term`, `coef`, `std_err`, `t`, `p_value`, `nobs`, `r2`, `adj_r2`.

When to use:

- To estimate a simple linear relationship (e.g., “how does y move with x ?”).
- To build a baseline regression before adding more complexity.

How to interpret the output (sophomore level):

- **coef**: estimated change in y for a one-unit increase in the predictor, holding other included predictors fixed.
- **std_err**: uncertainty in the coefficient estimate (smaller means more precise).
- **t** and **p_value**: measure how strongly the data rejects “coefficient = 0” under the model assumptions.
- **r2** / **adj_r2**: descriptive fit metrics (how much variation in y is explained in-sample).

Common pitfalls (especially in time series):

- **Correlation vs causation**: OLS alone does not identify causal effects.
- **Autocorrelation**: time series errors are often correlated over time, which can make naive standard errors misleading.
- **Nonstationarity**: regressing trending series on trending series can produce “spurious regression” results.

Non-technical intuition

OLS is the standard “line of best fit” idea generalized to multiple predictors. It is often the first model you run to quantify relationships, but interpretation requires care (especially with time series).

Where: `src/finrec/recipes/econ/ols.py`

```
X2 = sm.add_constant(X, has_constant="add") if add_const else X
model = sm.OLS(y2, X2)
res = model.fit()
out = pd.DataFrame({
    "term": res.params.index.astype(str),
    "coef": res.params.values,
    "p_value": res.pvalues.values,
})
```

10.5.2 AR1Recipe

What it is: AR(1) via OLS: $y_t = c + \phi y_{t-1} + \varepsilon_t$.

When to use:

- To measure **persistence**: does today’s value mostly carry over from yesterday/month/quarter?
- As a simple baseline time-series model (many series are highly persistent).

Interpretation:

- If ϕ is close to 1, the series is very persistent (shocks die out slowly).
- If ϕ is close to 0, the series has little persistence (next period is mostly new noise).

Common pitfalls:

- If the series has a strong trend, AR(1) on levels can be misleading; consider differencing or growth rates.
- The usual interpretation assumes a stable data-generating process; structural breaks can change persistence.

Non-technical intuition

AR(1) answers: “how much does the past predict the present?” It is the simplest way to quantify persistence in a time series.

Where: `src/finrec/recipes/econ/ar1.py`

```
y_lag = y.shift(1)
data = pd.concat([y.rename("y"), y_lag.rename("y_lag1")], axis=1).dropna()
Y = data["y"]
X = sm.add_constant(data["y_lag1"], has_constant="add")
res = sm.OLS(Y, X).fit()
```

10.5.3 LocalProjectionIRFRecipe

What it is: For each horizon h : regress y_{t+h} on shock at t and optional controls. IRF at h = coefficient on shock. Confidence intervals via $\hat{\beta} \pm z \cdot \text{se}$.

When to use:

- When you want to study dynamics: how does y respond over time after a one-time shock?
- When you want an IRF-like object without fully specifying a VAR.

Interpretation (plain English): the IRF at horizon h is “the predicted change in y h periods after the shock.” If the confidence interval includes 0, the response is not statistically distinguishable from zero under the model assumptions.

Common pitfalls:

- **Identification:** the shock variable must be plausibly exogenous for a causal interpretation.
- **Small samples:** as h grows, you lose observations (because y_{t+h} shifts forward), so intervals widen.
- **Controls:** including or excluding controls can change the estimated response substantially.

Non-technical intuition

Local projections produce an impulse response function by running a sequence of regressions. It is a simple way to visualize “what happens next” after a shock, horizon by horizon.

Where: `src/finrec/recipes/econ/lp_irf.py`

```
rows = []
for h in range(horizons + 1):
    y_fwd = y.shift(-h)
    data = pd.concat([y_fwd.rename("y_fwd"), shock.rename("shock")], axis=1).dropna()
    res = sm.OLS(data["y_fwd"], sm.add_constant(data["shock"])).fit()
    rows.append({"horizon": h, "irf": float(res.params["shock"])})
```


10.6 News Processing

10.6.1 FinBERTDailySentimentRecipe

What it is: FinBERT sentiment on title+snippet; aggregates to daily mean, sum, n_articles, pos_share, neg_share.

When to use:

- When you want a simple quantitative proxy for “news tone” about a topic, company, or macro keyword.
- When you want to align text-based information with numeric time series (prices, inflation, rates).

How to interpret the output:

- **n_articles:** how much coverage there was that day (an “attention” measure).
- **pos_share / neg_share:** fraction of articles classified as positive/negative.
- **mean / sum:** average or total signed sentiment score for the day (implementation-specific scaling).

Common pitfalls:

- **Model limitations:** FinBERT is trained on financial text, but it can still misread sarcasm, context, or mixed sentiment.
- **Selection:** the set of articles you get depends heavily on the query; sentiment is conditional on what you asked for.
- **Not “truth”:** sentiment is a noisy measurement, best used as an exploratory feature rather than a definitive signal.

Non-technical intuition

Turns text into a time series: score each article as positive/negative, then summarize the day. This lets you plot “news tone” next to economic variables, just like any other series.

Where: src/finrec/recipes/news/sentiment_finbert.py

```
pipe = transformers.pipeline("sentiment-analysis", model="ProsusAI/finbert")
preds = pipe(texts) # texts = title + snippet
scored = _map_labels_to_signed_scores(preds)
agg = scored.groupby("date", as_index=False).agg(
    n_articles=("label", "count"),
    sentiment_mean=("signed", "mean"),
)
```

10.7 Technical Indicators

10.7.1 SimpleMovingAverageRecipe

What it is: $SMA_t = \frac{1}{w} \sum_{i=0}^{w-1} x_{t-i}$.

When to use:

- To smooth a noisy series (especially prices) and visualize underlying trends.

- As a simple feature in exploratory analysis (e.g., comparing price to its moving average).

Interpretation: SMA is the average of the last w observations. Larger w means more smoothing but more lag.

Non-technical intuition

SMA is like “blur” for a noisy line: it reduces day-to-day wiggles so you can see the broader movement more clearly.

Where: `src/finrec/recipes/technical/sma.py`

```
x = pd.to_numeric(df[value_col], errors="coerce")
df2[out_col] = x.rolling(window=window, min_periods=max(2, window // 3)).
    mean()
```

10.7.2 ExponentialMovingAverageRecipe

What it is: EMA via `ewm(span=span).mean()`.

When to use:

- When you want smoothing like SMA but with more weight on recent observations.
- When you care about reacting faster to changes (EMA typically lags less than SMA for the same window size).

Interpretation: EMA is a weighted average where weights decline as you go back in time.

Non-technical intuition

EMA is a moving average that “pays more attention” to recent data, so it adapts faster when the series changes direction.

Where: `src/finrec/recipes/technical/ema.py`

```
x = pd.to_numeric(df[value_col], errors="coerce")
df2[out_col] = x.ewm(span=span, adjust=False, min_periods=max(2, span // 3)
    ).mean()
```

10.7.3 RSIRecipe

What it is: $RSI = 100 - \frac{100}{1+RS}$, $RS = \frac{\text{avg gain}}{\text{avg loss}}$.

When to use:

- To summarize recent up vs down moves (a momentum/strength indicator).
- To create a bounded 0–100 feature that is easy to compare across assets.

Interpretation: high RSI means recent gains dominate losses; low RSI means recent losses dominate gains. Common “overbought/oversold” thresholds are heuristic, not a law of nature.

Non-technical intuition

Momentum indicator 0–100; high = overbought, low = oversold.

Where: `src/finrec/recipes/technical/rsi.py`

```

delta = s.diff()
gain = delta.clip(lower=0)
loss = (-delta).clip(lower=0)
avg_gain = gain.ewm(alpha=1/window, adjust=False, min_periods=window).mean()
avg_loss = loss.ewm(alpha=1/window, adjust=False, min_periods=window).mean()
rsi = 100 - (100 / (1 + (avg_gain / avg_loss)))

```

10.7.4 MACDRecipe

What it is: MACD line = EMA(fast) - EMA(slow); Signal = EMA(MACD); Histogram = MACD - Signal.

When to use:

- To summarize whether short-run momentum is stronger/weaker than long-run momentum.
- To visualize changes in trend and potential turning points (descriptive, not guaranteed predictive).

Interpretation:

- MACD > 0 means the fast EMA is above the slow EMA (upward momentum relative to the long-run average).
- The histogram shows whether momentum is accelerating or decelerating.

Non-technical intuition

MACD compares a short-run and long-run moving average. It is a compact way to describe trend vs momentum in one plot.

Where: src/finrec/recipes/technical/macd.py

```

ema_fast = s.ewm(span=fast, adjust=False).mean()
ema_slow = s.ewm(span=slow, adjust=False).mean()
macd_line = ema_fast - ema_slow
signal_line = macd_line.ewm(span=signal, adjust=False).mean()
hist = macd_line - signal_line

```

10.7.5 RollingVolatilityRecipe

What it is: Rolling std of returns; optionally annualized with $\sigma \sqrt{\text{periods_per_year}}$.

When to use:

- To measure how “risky” or variable a series has been recently.
- To compare volatility across assets or across time (especially when annualized consistently).

Interpretation: higher rolling volatility means returns have been fluctuating more within the chosen window. Annualization puts volatility in “per year” units (e.g., daily returns scaled by $\sqrt{252}$).

Non-technical intuition

Volatility is a measure of variability. Rolling volatility asks: “over the last w periods, how much do returns bounce around?” Big volatility often corresponds to turbulent markets.

Where: `src/finrec/recipes/technical/rolling_vol.py`

```
r = p.pct_change()
vol = r.rolling(window=window, min_periods=max(2, window // 3)).std(ddof=0)

if annualize:
    vol = vol * math.sqrt(periods_per_year)
```

10.8 Time Series Transforms**10.8.1 LogReturnsRecipe**

What it is: $\ln(p_t) - \ln(p_{t-1})$.

When to use:

- For many finance tasks, returns are more stable to model than price levels.
- When you want changes that add over time: log returns are approximately additive across periods.

Interpretation: for small changes, $\ln(p_t) - \ln(p_{t-1})$ is approximately the percent change in **decimal** form. Multiply by 100 to get percent units (roughly).

Common pitfalls:

- Prices must be positive. If $p_t \leq 0$, logs are undefined.
- If your series is already a rate (like unemployment), log returns may not be meaningful.

Where: `src/finrec/recipes/timeseries/log_returns.py`

```
s = pd.to_numeric(df[value_col], errors="coerce")
logp = s.apply(lambda x: math.log(x) if pd.notna(x) and x > 0 else float("nan"))
df2[out_col] = logp.diff()
```

10.8.2 RollingZScoreRecipe

What it is: $z_t = \frac{x_t - \mu_t}{\sigma_t}$ with rolling μ_t, σ_t .

When to use:

- To express values relative to their recent history (“how unusual is today compared to the last year?”).
- To standardize series for comparison across different units/scales.

Interpretation: a z-score of +2 means the value is 2 rolling standard deviations above its rolling mean. Negative values are below-average.

Common pitfalls:

- Rolling estimates depend on window choice; short windows react quickly but are noisy.

- If σ_t is near zero (almost no variation), z-scores can blow up.

Where: `src/finrec/recipes/timeseries/rolling_zscore.py`

```
mu = x.rolling(window=window, min_periods=max(2, window // 3)).mean()
sd = x.rolling(window=window, min_periods=max(2, window // 3)).std(ddof=0)
df2[f"{prefix}_mean"] = mu
df2[f"{prefix}_std"] = sd
df2[f"{prefix}_z"] = (x - mu) / sd
```

10.9 Infrastructure (continued)

10.9.1 RecipeRegistry

What it is: Singleton with all 22 recipes; lookup by `recipe_id`.

Why it exists: recipes are the building blocks for transforms, models, and forecasts. A central registry lets the UI list available recipes and submit runs by ID.

How to extend (developer view): implement a recipe (see 10.9.2) and register it during registry initialization.

Non-technical intuition

The registry is a menu. The UI asks: “what recipes exist?” and the registry answers with a list it can display and run by ID.

Where: `src/finrec/recipes/registry.py`

```
def get_recipe_registry() -> RecipeRegistry:
    global _RECIPE_REGISTRY
    if _RECIPE_REGISTRY is None:
        reg = RecipeRegistry(_recipes={})
        reg.register(LogReturnsRecipe())
        reg.register(SimpleMovingAverageRecipe())
        # ... register the rest ...
        _RECIPE_REGISTRY = reg
    return _RECIPE_REGISTRY
```

10.9.2 Recipe base protocol

What it is: Recipe has meta: `RecipeMeta` and `run(df, params, ctx) -> DataFrame`.

Interpretation: recipes are pure transformations: they take an input dataset (typically a CSV artifact loaded into a `DataFrame`) plus parameters, and return an output dataset. This makes it easy to chain steps: output artifacts become inputs to later recipes.

Non-technical intuition

A recipe is like a “function you can run from the UI.” Input: a dataset + settings. Output: a new dataset you can save, plot, and use later.

Where: `src/finrec/recipes/base.py`

```
@dataclass(frozen=True)
class RecipeMeta:
    id: str
    name: str
    description: str
```

```
class Recipe(Protocol):
    meta: RecipeMeta
    def run(self, df: pd.DataFrame, params: dict, ctx: JobContext) -> pd.
        DataFrame: ...
```

10.9.3 SQLiteStorage

What it is: Tables `jobs`, `job_logs`. Methods: `create_job`, `set_status`, `set_output_path`, `append_log`, `list_jobs`, `list_logs`.

How to interpret the database:

- `jobs` records **what was requested** (provider/recipe and request JSON), **current status**, and where the output CSV lives.
- `job_logs` records timestamped messages that are helpful for debugging (errors, retries, etc.).

Non-technical intuition

SQLiteStorage is the app's memory. It keeps a permanent record of what you ran and whether it worked, plus where to find the output files.

Where: `src/finrec/storage/sqlite.py`

```
def create_job(self, job_id, *, kind, provider_kind, provider_id, request):

    now = _utc_now_iso()
    with self._connect() as conn:
        conn.execute(
            "INSERT INTO jobs (job_id, created_at, updated_at, status,
            kind, provider_kind, provider_id, request_json) VALUES (?, ?, ?, ?, ?,
            ?, ?, ?);",
            (job_id, now, now, "QUEUED", kind, provider_kind, provider_id,
            json.dumps(request)),
        )
```

10.9.4 JobRunner

What it is: `ThreadPoolExecutor(max_workers=2)`. On success writes `{job_id}.csv`; on failure sets status `FAILED` and stores error.

Why it exists: it decouples the UI from work. The Streamlit UI can submit a job quickly and then refresh while the job runs in the background.

Common pitfalls:

- Threaded execution is good for I/O (API calls) but is not a full distributed job system. Long CPU-heavy jobs can still feel slow.
- Outputs are CSV artifacts; if you need large-scale storage or caching, you would redesign this layer.

Non-technical intuition

JobRunner is the worker that actually does the work. It runs jobs in the background, writes outputs to CSV, and updates statuses so the UI can show progress.

Where: `src/finrec/jobs/runner.py`

```
def _run() -> None:
    try:
        self.storage.set_status(job_id, "RUNNING")
        df = fn(ctx)
        out_path = self.results_dir / f"{job_id}.csv"
        df.to_csv(out_path, index=False)
        self.storage.set_output_path(job_id, out_path.as_posix())
        self.storage.set_status(job_id, "SUCCEEDED")
    except Exception as e:
        self.storage.set_status(job_id, "FAILED")
        self.storage.set_error(job_id, f"{e}\n{traceback.format_exc()}")
```

10.9.5 Pipeline submission

What it is: `submit_provider_fetch`, `submit_recipe_run`, `get_jobs_by_id` in `src/finrec/ui/pipelines.py`

Interpretation: these functions are the glue between UI controls and the job runner. The UI never calls a provider or recipe directly; it submits a job and then reads the artifact when the job finishes.

Non-technical intuition

Pipeline functions translate button clicks into jobs. That keeps the UI simple and keeps all execution in one standardized place.

Where: `src/finrec/ui/pipelines.py`

```
def submit_provider_fetch(*, runner: JobRunner, kind: str, provider_id:
    str, request: dict) -> str:
    reg = get_registry()
    provider = reg.get(kind, provider_id)
    def job_fn(ctx):
        return provider.fetch(request, ctx=ctx)
    return runner.submit(kind="provider_fetch", provider_kind=kind,
        provider_id=provider_id, request=request, fn=job_fn)
```

10.10 Datasets

10.10.1 build_merged_dataset

What it is: Merges CSVs on date; optional forward-fill. Inputs: `path`, `date_col`, `value_col`, `alias`.

When to use:

- Before regressions: you usually need all variables in one table with a shared date index.
- When combining macro and finance series with different frequencies (carefully).

Common pitfalls:

- **Merge type:** an outer merge keeps all dates and introduces missing values; an inner merge keeps only overlapping dates.
- **Forward-fill:** can be useful for mixing frequencies (e.g., monthly macro with daily prices), but it also embeds an assumption (macro value stays constant until next release).

Non-technical intuition

Merging is how you turn “separate series” into a regression-ready table. The key choice is how you handle missing dates: drop them (inner) or keep them and fill gaps (outer + optional forward-fill).

Where: `src/finrec/datasets/merge.py`

```
merged = None
for spec in inputs:
    df = pd.read_csv(spec["path"])
    s = pd.DataFrame({"date": pd.to_datetime(df[spec["date_col"]]).dt.date.
                     astype(str)})
    s[spec["alias"]] = pd.to_numeric(df[spec["value_col"]], errors="coerce")
    merged = s if merged is None else merged.merge(s, on="date", how=
    merge_how)
if ffill:
    merged = merged.set_index("date").sort_index().ffill().reset_index()
```

10.11 Visualization**10.11.1 plot_timeseries**

What it is: Plotly chart with secondary axis, log scaling, recession shading via `add_vrect`.

How to read the chart options:

- **Secondary axis:** use when two series have very different scales (e.g., CPI level vs inflation rate).
- **Log scale:** useful for level series that grow over time; changes become comparable in percentage terms.
- **Recession shading:** uses NBER recession dates (via USREC) to provide macro context.

Non-technical intuition

Plotting is part of analysis. The app’s plotter focuses on the most common time-series needs: multiple lines, different scales, and contextual recession shading.

Where: `src/finrec/viz/plotly_timeseries.py`

```
fig = subplots.make_subplots(specs=[[{"secondary_y": bool(right_cols)}]])
for c in left_cols:
    fig.add_trace(go.Scatter(x=plot_df[x_col], y=plot_df[c], mode="lines",
                             name=str(c), secondary_y=False))
for c in right_cols:
    fig.add_trace(go.Scatter(x=plot_df[x_col], y=plot_df[c], mode="lines",
                             name=str(c), secondary_y=True))
if recession_intervals:
    for (start, end) in recession_intervals:
        fig.add_vrect(x0=start, x1=end, fillcolor="rgba(120,120,120,0.20)",
                      line_width=0)
```


10.11.2 build_recession_intervals

What it is: Converts USREC 0/1 series into (start, end) intervals for shading.

Interpretation: USREC is a FRED recession indicator (1 during recession months). This helper just turns the 0/1 sequence into date ranges so the plotter can shade those intervals.

Where: src/finrec/viz/plotly_timeseries.py

```
intervals = []
in_rec, start = False, None
for row in s.itertuples():
    if row.flag == 1 and not in_rec:
        in_rec, start = True, row.dt
    if row.flag == 0 and in_rec:
        intervals.append((start, last_date))
    in_rec = False
```

Non-technical intuition

Recession shading is a visual aid: it helps you see whether spikes or breaks in the data line up with recessions.

10.12 UI & Config

10.12.1 Time series catalog

What it is: build_finance_options, build_econ_options — curated tickers/FRED IDs with recents and custom.

Why it exists: for class projects, the hardest part is often “which series should I even use?” The catalog provides a curated default list (plus your recents and custom entries) so you can start quickly.

Non-technical intuition

The catalog is a set of sensible defaults. You can always type your own ticker/FRED ID, but the catalog prevents “blank page” syndrome.

Where: src/finrec/ui/timeseries_catalog.py

```
def build_finance_options(recent_ids: List[str], custom_ids: List[str]) ->
    List[SeriesOption]:
    return _build_options(_FINANCE_CATALOG, _FINANCE_BY_ID,
        _FINANCE_CATEGORY_PRIORITY, recent_ids, custom_ids)

def build_econ_options(recent_ids: List[str], custom_ids: List[str]) ->
    List[SeriesOption]:
    return _build_options(_ECON_CATALOG, _ECON_BY_ID,
        _ECON_CATEGORY_PRIORITY, recent_ids, custom_ids)
```

10.12.2 AppConfig / load_config

What it is: Loads FINREC_DB_PATH, FINREC_RESULTS_DIR from .env.

Interpretation: configuration is intentionally minimal: where to store the database and where to write artifacts. The defaults are good for local work; changing paths is helpful if you want outputs in a shared folder.

Non-technical intuition

Config answers: “where are my job records stored, and where do my CSV outputs go?” Everything else is kept simple on purpose.

Where: `src/finrec/config.py`

```
def load_config() -> AppConfig:
    load_dotenv(override=False)
    db_path = Path(os.getenv("FINREC_DB_PATH", "data/finrec.db"))
    results_dir = Path(os.getenv("FINREC_RESULTS_DIR", "data/results"))
    db_path.parent.mkdir(parents=True, exist_ok=True)
    results_dir.mkdir(parents=True, exist_ok=True)
    return AppConfig(db_path=db_path, results_dir=results_dir)
```

10.12.3 Optional dependency helpers

What it is: `optional_import`, `require_optional` — safe imports with install hints.

Why it exists: it keeps the base install small while still supporting advanced features. Instead of crashing with a confusing `ModuleNotFoundError`, the app can tell you exactly which extra to install.

Non-technical intuition

Optional dependencies let the project be both lightweight and capable: beginners can run the basics, and advanced features “turn on” when you install the extra packages.

Where: `src/finrec/providers/utils/optional.py`

```
def optional_import(module_name: str) -> Optional[ModuleType]:
    try:
        return importlib.import_module(module_name)
    except Exception:
        return None

def require_optional(module_name: str, extra_hint: str) -> ModuleType:
    mod = optional_import(module_name)
    if mod is None:
        raise ImportError(f"Optional dependency '{module_name}' is not installed. Install with: pip install -e \".[{extra_hint}]\")
    return mod
```

10.12.4 Main Streamlit app

What it is: `streamlit_app.py::main()` — Finance/Econ modes, query UI, results, dataset builder, auto-refresh.

How to think about the UI: it is essentially a front-end for submitting jobs and then visualizing/downloading artifacts. Once you understand jobs and artifacts, the UI is straightforward.

Non-technical intuition

Streamlit is the front door. Its job is to collect inputs, submit work, and display outputs—while the pipeline does the heavy lifting.

Where: streamlit_app.py

```
def _get_singletons():
    if "finrec_config" not in st.session_state:
        st.session_state.finrec_config = load_config()
    if "finrec_storage" not in st.session_state:
        storage = SQLiteStorage(st.session_state.finrec_config.db_path)
        storage.init_schema()
        st.session_state.finrec_storage = storage
    if "finrec_runner" not in st.session_state:
        st.session_state.finrec_runner = JobRunner(st.session_state.
            finrec_storage, st.session_state.finrec_config.results_dir)
```

10.12.5 Legacy pages

What it is: 01_Data_Pull, 02_Jobs, 03_Preview, 04_Analysis, 05_Datasets.

Why they still matter: these pages show the same workflow split into steps. They can be easier for a new contributor to understand because each page focuses on one concept (fetch, jobs, preview, analysis, merge).

Non-technical intuition

If the main app feels like “too much at once,” the legacy pages provide a step-by-step tutorial version of the same pipeline.

Where: legacy_pages/

```
recipes = recipe_reg.list()
recipe_label_to_id = {f"{r.meta.name} ({r.meta.id})": r.meta.id for r in
    recipes}
recipe_label = st.selectbox("Recipe", list(recipe_label_to_id.keys()))
recipe_id = recipe_label_to_id[recipe_label]
recipe = recipe_reg.get(recipe_id)
```

10.12.6 E2E smoke script

What it is: scripts/e2e_smoke.py — runs market fetch, news fetch, SMA, FRED, inflation_yoy, AR1; asserts output columns.

Why it exists: it is an automated “does the whole pipeline basically work?” check. If providers or recipes break due to dependency/API changes, the smoke script should fail quickly and point to the issue.

Non-technical intuition

The smoke test is a quick health check: it runs a realistic sequence of jobs and confirms the expected output columns exist.

Where: scripts/e2e_smoke.py

```
market_job = submit_provider_fetch(
    runner=runner, kind="market", provider_id=market_provider,
    request={"symbol": "AAPL", "start_date": "2020-01-01", "end_date": "
    2020-02-01", "n": 30},
)
```

11 Completeness Checklist

Item	Details
Files scanned	src/finrec/, streamlit_app.py, legacy_pages/, scripts/, tests/, pyproject.toml, .env.example
Features counted	43 (4 providers, 22 recipes, 6 infra, 4 orchestration/storage/datasets, 2 viz, 4 UI/config/scripts)
Inventory ↔ sections	Every row in the feature index has a corresponding section.
Snippets per feature	Every feature section includes at least one code snippet or intuition box.
Potentially missed	__init__.py, egg-info/, LaTeX build artifacts, .pytest_cache/ excluded by design.